

Arduino 2 — Analog Input and Serial Output

Tutorial Group: Arduino

Goal

Extend the Blink program from *Arduino 1 — Install the Arduino IDE and Run the Blink Test* to:

- Send information from the Arduino to the computer using the Serial Monitor.
- Read a variable analog voltage using analog input A0.
- Understand the relationship between input voltage and the Mega 2560's ADC reading.
- Convert the ADC reading into a voltage.
- Use the ADC reading to control the LED blink rate.

This tutorial introduces the basic **input → process → output** pattern used throughout embedded systems.

What You Need

- Arduino Mega 2560
- USB cable
- Computer with the Arduino IDE
- Breadboard and jumper wires
- Potentiometer (variable resistor) from the Mega 2560 evaluation kit

Part 1 — Serial Output

The USB connection between the Arduino and computer can be used to send information from the Arduino back to the computer.

The Arduino IDE's **Serial Monitor** can display this information.

Modify the Blink program from *Arduino 1 — Install the Arduino IDE and Run the Blink Test* so that it also sends a message to the Serial Monitor approximately once per second.

You will need to:

- Start Serial communication in `setup()`.
- Print a message from `loop()`.
- Use the Serial Monitor to view the result.

The functions you will need are:

```
Serial.begin()  
Serial.print()  
Serial.println()
```

Arduino's Serial library provides a simple way for a program running on the Arduino to send text and numbers to the computer through the USB connection. The Serial Monitor in the Arduino IDE can then display that information.

The most common functions you will use at this point are `Serial.begin()`, `Serial.print()`, and `Serial.println()`. `Serial.begin()` starts the serial connection and specifies the communication speed (baud rate). `Serial.print()` sends an item without moving to a new line. `Serial.println()` sends an item and then moves to a new line.

Important: `Serial.print()` and `Serial.println()` print only one item per invocation. Multiple calls are needed when you want to combine text, variables, and other values into one output line. This is different from the `printf()` style output you may have used in standard C. Instead of giving one `printf()` call a format string containing several values, Arduino's `Serial.print()` and `Serial.println()` methods normally print one item at each invocation. To create a complicated output line, you therefore use multiple print or `println` calls.

For example:

```
Serial.begin(9600);
Serial.println("Hello from the Arduino");
Serial.print("Voltage = ");
Serial.print(voltage);
Serial.print("v, ADC = ");
Serial.println(adcValue);
```

Arduino's Serial library provides a simple way for a program running on the Arduino to send text and numbers to the computer through the USB connection.

The first statement starts serial communication at 9600 baud. The second sends the text Hello from the Arduino to the Serial Monitor and then moves to the next line. The next four lines print both the ADC value and the calculated voltage.

This tutorial uses 9600 baud for simplicity. Later tutorials may use 115200 baud.

Open:

Tools → Serial Monitor

and select the same 9600 baud rate to see the messages.

Change the message and the delay time.

Observe how changing the delay affects the rate at which messages appear in the Serial Monitor.

Consider:

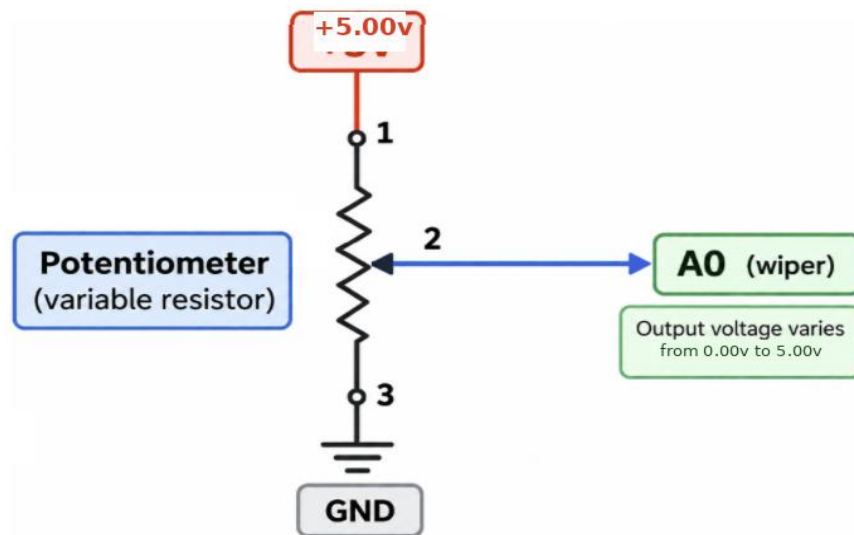
- Why does `Serial.begin()` belong in `setup()`?
- Why does `Serial.println()` belong in `loop()`?
- What happens if you remove the delay?

Part 2 — Connect a Variable Resistor

The Mega 2560 evaluation kit includes a **potentiometer**, also called a variable resistor.

A potentiometer has three terminals. The two outside terminals are the ends of a resistive element, and the center terminal is a movable **wiper**. By turning the potentiometer, the wiper moves along the resistive element and produces a variable voltage.

For this tutorial, connect the potentiometer as follows:



Connections:

- One outside terminal → +5.00v
- Center/wiper terminal → A0
- Other outside terminal → GND

If the voltage changes in the opposite direction from what you expect when turning the potentiometer, swap the two outside connections.

As you turn the potentiometer, the voltage at A0 should vary from approximately **0.00v to 5.00v**.

Part 3 — Read A0

The Mega 2560 contains a 10-bit analog-to-digital converter.

The Arduino function used to read an analog input is:

```
analogRead()
```

Modify your Serial program so that it repeatedly reads A0, calculates the corresponding voltage, and then displays the results on the Serial Monitor.

For this tutorial, declare both variables before the program begins. Inside loop(), read A0 first, calculate the voltage second, and only then print the results:

```
int adcValue;  
float voltage;  
  
adcValue = analogRead(A0);  
voltage = adcValue * (5.0 / 1023.0);
```

The first line declares an integer for the 10-bit ADC reading. The second declares a floating-point variable for the calculated voltage. The first statement reads A0, and the second applies the ADC-to-voltage equation.

Your program should:

- Read A0.
- Calculate the voltage from the ADC reading.
- Send the ADC value and calculated voltage to the Serial Monitor.
- Wait briefly.
- Repeat.

Turn the potentiometer while watching the Serial Monitor.

You should see the value change as you turn the potentiometer.

The Mega 2560's 10-bit ADC produces values from:

0 to 1023

These values are approximate because the actual 5.00v supply and ADC reference may not be exactly 5.00v.

Experiment - turn the potentiometer slowly from one end to the other.

Observe:

- The minimum ADC value.
- The maximum ADC value.
- The value near the middle of the potentiometer's rotation.

Do the measured values agree with what you expect?

Part 4 — Relate the ADC Value to Voltage

The Mega 2560 uses a 10-bit ADC, giving it:

$2^{10} = 1024$ possible values

represented by:

0 through 1023

Assuming a nominal 5.00v reference, the approximate relationship is:

$\text{Voltage} = \text{ADC value} \times 5.0 / 1023.0$

Use this relationship to calculate the expected voltage for several ADC readings.

For example, an ADC reading of approximately 512 corresponds to:

$512 \times 5.0 / 1023.0 \approx 2.50\text{v}$

Part 5 — Calculate and Display the Voltage

Now modify your program so that it displays both the ADC value **and the calculated voltage** on the Serial Monitor.

For example:

```
ADC: 512 Voltage: 2.50v
ADC: 511 Voltage: 2.50v
ADC: 513 Voltage: 2.51v
```

The exact values will depend on the actual voltage supplied to the Arduino. The ADC reading is an integer, but the calculated voltage will usually contain a fractional part.

For example:

```
2.50v
3.27v
4.81v
```

The Arduino C/C++ language provides the float data type for storing numbers that contain a fractional part.

For example:

```
float voltage;
```

Use a *float* variable to hold the calculated voltage.

Consider why the calculation uses:

```
5.0
1023.0
```

rather than:

```
5
1023
```

The distinction is related to **integer arithmetic versus floating-point arithmetic** in C/C++.

If you are not familiar with integer and floating-point arithmetic, investigate the difference before continuing.

Experiment - Turn the potentiometer slowly and observe both the ADC value and calculated voltage.

Verify that:

- The ADC value increases as the voltage increases.
- The calculated voltage increases as the ADC value increases.
- The calculated voltage is approximately what you expect from the potentiometer position.

Part 6 — Use A0 to Control the Blink Rate

Return to the Blink program from *Arduino 1 — Install the Arduino IDE and Run the Blink Test*.

Previously, the blink rate was controlled by fixed values such as:

```
delay(1000);
```

Modify the program so that the delay is determined by the ADC value from A0.

The finished program should behave as follows:

- Changing the potentiometer position changes the LED blink rate.
- The Arduino `map()` function converts a value from one range of numbers to another range.

The general form is:

```
map(value, fromLow, fromHigh, toLow, toHigh)
```

For example:

```
delayTime = map(adcValue, 0, 1023, 1000, 100);
```

This converts an ADC reading from the range **0–1023** into a delay range of **1000–100 milliseconds**.

In this tutorial:

- An ADC reading of **0** produces a delay of **1000 ms**.
- An ADC reading of **1023** produces a delay of **100 ms**.
- Values between 0 and 1023 are scaled proportionally between those limits.

The `map()` function does not automatically know which direction the conversion should go. Reversing the output values reverses the relationship:

```
delayTime = map(adcValue, 0, 1023, 100, 1000);
```

Now a higher ADC reading produces a longer delay.

For this project, use `map()` to convert the potentiometer's ADC reading into a delay value that produces the desired LED blink rate.

You will need to decide:

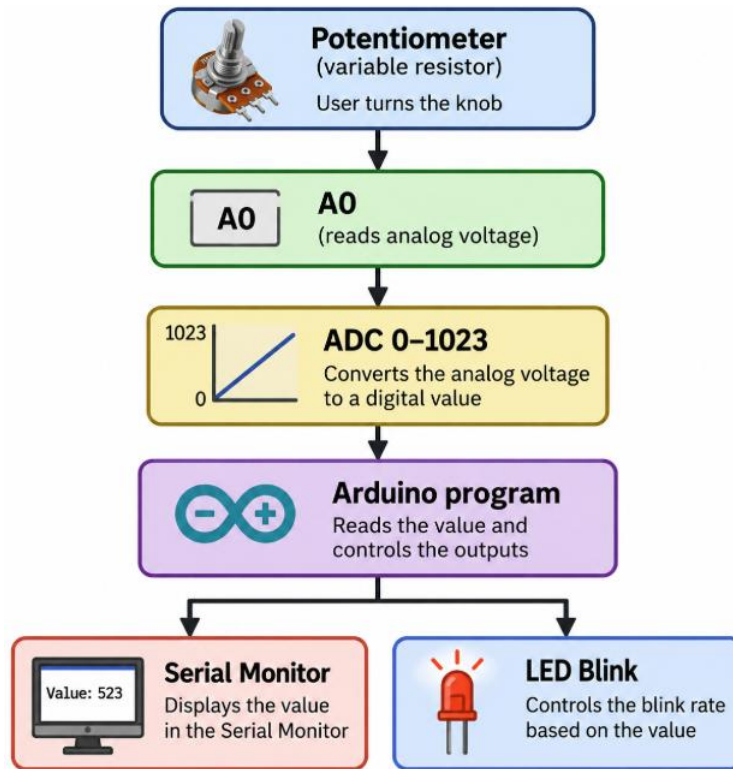
- What is the slowest desired blink rate?
- What is the fastest desired blink rate?
- Should increasing voltage make the LED faster or slower?

What You Should Understand

At the completion of this tutorial, you should understand:

- How to send information from the Arduino to a computer using Serial communication.
- Why `Serial.begin()` belongs in `setup()`.
- How `Serial.println()` can be used to display program information.
- How a potentiometer can produce a variable voltage.
- How the Mega 2560's ADC converts an analog voltage into a number.
- Why the ADC produces values from 0–1023.
- How to convert an ADC reading into an approximate voltage.
- The difference between integer and floating-point values.
- Why a float is useful for storing a calculated voltage.
- How an input value can be used to control an output.
- How to map one numerical range into another.

The overall system you have created is:



Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I can send information from the Arduino to the Serial Monitor.
- ☐ I can connect a potentiometer to provide a variable voltage to A0.
- ☐ I can read analog input A0.
- ☐ I understand the 0–1023 ADC range.
- ☐ I can calculate and display the voltage corresponding to an ADC reading.
- ☐ I understand why a float is appropriate for the calculated voltage.
- ☐ I can use the ADC value to control the LED blink rate.
- ☐ I can explain the basic input → process → output operation of my program.

If you can complete these tasks and explain what your program is doing, you are ready for the next tutorial.